PROVISIONAL PATENT APPLICATION

United States Patent and Trademark Office

TITLE OF THE INVENTION

System and Method for Process Ancestry Chain-Scoped Endpoint Security Enforcement Utilizing a Two-Tier Evaluation Engine with In-Memory Ancestry Acceleration

---

CROSS-REFERENCE TO RELATED APPLICATIONS

This application is a provisional application filed under 35 U.S.C. Section 111(b). No priority is claimed from any prior filing.

Docket Number: [TO BE ASSIGNED]

ABSTRACT

A system and method for real-time endpoint security enforcement wherein user-configurable allow and block rules are scoped by process ancestry chain patterns. The invention introduces a domain-specific pattern language for specifying hierarchical process ancestry constraints using directional token separators, single-process wildcards, multi-process wildcards, and operating system user identity prefixes. A two-tier evaluation engine compiles unscoped rules into constant-time lookup structures including hash sets and prefix lists for microsecond-latency inline evaluation, while automatically segregating rules bearing ancestry chain filters into a separate evaluation path that reconstructs the live process lineage from a dual-source in-memory ancestry index. The ancestry index is bootstrapped from a local graph database and incrementally maintained through process lifecycle callbacks, enabling sub-millisecond chain reconstruction without graph query latency. Upon match, the engine synthesizes a deterministic security finding and triggers an immediate automated response action, bypassing asynchronous machine-learning-based analysis. This architecture resolves the Exception Tuning Paradox inherent in existing endpoint detection and response systems that rely on flat, context-free indicator lists.

---

1. BACKGROUND OF THE INVENTION

1.1 Field of the Invention

The present invention relates generally to endpoint detection and response (EDR) systems in cybersecurity, and more specifically to systems and methods for real-time inline enforcement of security rules that incorporate process ancestry chain context when making allow or block decisions at the operating system event level.

1.2 Description of Related Art

Endpoint detection and response systems monitor operating system telemetry events -- including process creation, network connections, file operations, and domain name resolution -- to detect and respond to security threats. A critical operational requirement of such systems is the ability for security operators to define exception rules (allowlists and blocklists) that tune enforcement behavior to the specific operational environment.

Several prior art systems address aspects of process ancestry analysis and rule-based enforcement, but each exhibits fundamental limitations that the present invention overcomes.

U.S. Patent Application Publication No. US 2019/0266323 A1 (CrowdStrike Technology, Inc., published September 5, 2019, now abandoned) discloses a system for identifying suspicious activity patterns based on ancestry relationships. The disclosed system identifies a trigger command in a running process, identifies an ancestry command associated with the trigger command, determines an ancestry level of the ancestry command, and upon determining that the ancestry level differs from an expected ancestry level, identifies a suspicious pattern. However, this system is limited to detection and alerting -- it does not disclose any mechanism for user-configurable allow or block rules that are scoped by process ancestry chain patterns. The rules in the disclosed system are vendor-defined behavioral indicators, not operator-configurable enforcement policies with chain context.

U.S. Patent Application Publication No. US 2017/0195350 A1 (Palo Alto Networks, Inc. and Cyber Secdo Ltd., published July 6, 2017) discloses a system for causality identification and attribution determination of processes in a network. The system deploys monitoring agents that observe initiated processes and determine whether a process was initiated at boot or by another process, identifying suspicious processes that lack expected initiation patterns. This system focuses on process classification heuristics and does not disclose operator-configurable allow or block rules, much less rules scoped by hierarchical process ancestry chain patterns with wildcard matching semantics.

U.S. Patent No. US 11,609,988 B2 (Acronis International GmbH, granted March 21, 2023) discloses systems and methods for detecting malicious behavior in process chains using artificial intelligence models. The system calculates suspicion levels for processes using pre-classified training data and restores affected objects from snapshots when malicious behavior is detected. While the system includes administrator-defined heuristic rules, these rules are updated by machine learning models and are not presented as user-configurable allow or block lists with process ancestry chain scoping. The system relies on probabilistic AI-based scoring rather than deterministic, user-tunable inline enforcement.

Existing commercial EDR products -- including CrowdStrike Falcon, SentinelOne Singularity, and Microsoft Defender for Endpoint -- provide exception mechanisms based on flat indicators of compromise (IOCs) such as file hashes (MD5, SHA-256), file system paths, process names, certificate signers, and IP addresses. These flat indicator lists lack process ancestry context entirely. While these systems internally track process parent-child relationships for asynchronous cloud-based detection and investigation (rendered as process trees in analyst interfaces), this graph context is not exposed to the inline allow/block rule evaluation engine.

1.3 The Exception Tuning Paradox

The limitation of flat, context-free indicator lists creates what the inventors term the "Exception Tuning Paradox" -- a forced trade-off between security coverage and operational usability that cannot be resolved within the constraints of existing exception architectures.

The Over-Broad Failure Mode. When an operator allowlists a process by name (for example, `curl`) to suppress false positives generated by legitimate administrative automation, the exception applies globally regardless of the process ancestry context. Consequently, `curl` cannot be blocked even when spawned as a descendant of a reverse-shell chain (for example, where a network listener spawns a shell interpreter, which in turn spawns `curl` to exfiltrate data). The allowlist creates a permanent blind spot.

The Under-Broad Failure Mode. When an operator blocklists a process by name (for example, `ncat`) to prevent lateral movement tooling, the block applies globally regardless of whether the invocation is malicious. Legitimate uses of the same binary -- such as invocations by configuration management frameworks (for example, Ansible spawning `ncat` for port testing) -- are disrupted. The blocklist generates persistent false positives that erode operator trust and may cause operational harm.

The root cause of both failure modes is identical: existing systems evaluate allow and block rules against isolated, flat attributes of the triggering event without considering the hierarchical process ancestry chain that produced the event.

Every exception is forced into a binary global state -- either the indicator is always allowed or always blocked, without regard to the execution context.

The present invention resolves this paradox by introducing a process ancestry chain filter that scopes any rule -- whether based on process name, IP address, domain name, CIDR range, or file path -- to fire only when the process ancestry chain of the triggering event matches a configurable hierarchical pattern. This enables operators to write precise exceptions such as "allow `ncat` only when spawned within an Ansible automation chain" or "block `bash` only when spawned as a child of `rsync`," eliminating both failure modes simultaneously.

A further technical challenge addressed by the present invention is the latency constraint inherent in inline enforcement. Existing systems that do track process ancestry do so asynchronously in cloud-based analytics pipelines, where latency on the order of seconds to minutes is acceptable. Inline enforcement at the operating system event level requires evaluation latency on the order of microseconds to low milliseconds to avoid degrading system performance. The present invention achieves this through a novel two-tier evaluation architecture with an in-memory ancestry acceleration index, described in detail below.

---

2. SUMMARY OF THE INVENTION

The present invention provides a system and method for endpoint security enforcement in which allow and block rules are scoped by process ancestry chain patterns and evaluated in real-time at the endpoint. The invention comprises three principal innovations:

First, a domain-specific pattern language for specifying process ancestry chain constraints. The pattern language uses a directional token separator (the greater-than character `>`) between process name elements, a single-asterisk wildcard (`*`) matching exactly one process in the chain, a double-asterisk wildcard (`**`) matching zero or more processes in the chain, and a user-identity prefix (`USER:`) scoping matches to a specific operating system user. Patterns are end-anchored at the terminal process of the chain and start-unanchored, permitting matching to begin at any ancestor position. Pattern matching is performed by a recursive backtracking algorithm that supports the full wildcard and anchoring semantics.

Second, a two-tier evaluation engine that achieves inline enforcement within the latency constraints of operating system event processing. In the first tier (the fast path), rules that do not specify a process ancestry chain filter are compiled into type-specific data structures optimized for constant-time or near-constant-time lookup, including hash sets for IP addresses and domain names, prefix lists for CIDR ranges, and ordered glob pattern lists for process names and file paths. In the second tier (the graph path), rules that specify a process ancestry chain filter are automatically segregated during compilation into a separate evaluation list; for each such rule, the engine reconstructs the process ancestry chain from a dual-source in-memory index and evaluates the rule only when both the chain filter pattern and the rule-type-specific pattern are satisfied.

Third, an in-memory process ancestry acceleration index that maintains four hash-map structures mapping process identifiers to parent process identifiers, child process identifier sets, graph node identifiers, and process names. The index is bootstrapped by a one-time scan of the local graph database and incrementally maintained through process lifecycle callbacks issued by the graph builder upon each process node insertion. This architecture enables sub-millisecond ancestry chain reconstruction without issuing graph database queries in the enforcement hot path.

By segregating evaluation paths at compilation time and utilizing an incrementally maintained in-memory ancestry index rather than recursive graph database queries, the present invention improves the functioning of the computer itself by reducing the CPU cycles required for inline security enforcement by orders of magnitude compared to graph-query-based approaches, eliminating disk I/O from the enforcement hot path, and enabling real-time process ancestry chain evaluation within the microsecond-to-millisecond latency budget imposed by operating system event processing

-- a latency constraint that prior art systems could not meet, forcing them to defer ancestry-aware analysis to asynchronous cloud pipelines.

---

3. DETAILED DESCRIPTION OF PREFERRED EMBODIMENTS

3.1 System Architecture Overview

The preferred embodiment implements a three-stage filtering and enforcement pipeline through which operating system telemetry events pass sequentially. Each stage operates at a different point in the event processing lifecycle and provides different levels of process ancestry context.

The telemetry event stream is sourced from kernel-level instrumentation hooks appropriate to the host operating system. On Linux, the preferred embodiment attaches Extended Berkeley Packet Filter (eBPF) programs to kernel tracepoints (specifically `tracepoint:syscalls:sys_enter_execve` for process execution and `kprobe:tcp_v4_connect` / `kretprobe:tcp_v4_connect` for network connections), capturing process identifiers, parent process identifiers, user identifiers, cgroup identifiers, executable paths, and command names directly from kernel data structures. The eBPF programs marshal captured fields to userspace via perf buffer ring buffers. As a fallback or supplement, the system attaches to the Linux Audit subsystem via a raw `AF_NETLINK` / `NETLINK_AUDIT` socket, installing audit rules for the `execve` and `connect` syscalls and parsing structured audit records from kernel netlink messages. On macOS, the system interfaces with the Endpoint Security Framework (ESF) for real-time process execution and file operation events, supplemented by Unified Log stream filtering on security-relevant subsystems and FSEvents for file system monitoring. On Windows, the system subscribes to Event Tracing for Windows (ETW) kernel providers including `Microsoft-Windows-Kernel-Process`, `Microsoft-Windows-Kernel-Network`, `Microsoft-Windows-Kernel-File`, and `Microsoft-Windows-Kernel-Registry` for process, network, file, and registry telemetry respectively. Network-layer telemetry is additionally captured via Berkeley Packet Filter (BPF) rules applied to TCP SYN packets, with TLS Server Name Indication (SNI) extraction and JA3 fingerprint computation performed on captured ClientHello messages.

Stage 1: Pre-Graph Allowlist Filter. The first stage operates after entity extraction but before events are written to the graph database. At this stage, extracted entities (processes, IP addresses, domains, file paths) are evaluated against allowlist rules using flat attribute matching only. Rules that specify a process ancestry chain filter are explicitly excluded from this stage, as chain context is not yet available. The purpose of this stage is to reduce graph write volume by dropping known-good operational noise (for example, operating system background daemons, antivirus scanner processes) before they accumulate in the graph. Entity removal at this stage cascades to all edges referencing the removed entity, and orphaned user nodes are cleaned up.

Stage 2: Fast-Path Synchronous Blocklist (The Core Invention). The second stage executes synchronously in the processor pipeline immediately after entity extraction and concurrently with or prior to graph insertion. This is where the two-tier evaluation engine operates. A matched rule at this stage triggers an immediate automated response action (for example, process termination via SIGKILL, generation of a critical-severity security finding) without waiting for asynchronous machine-learning-based analysis. The evaluation engine has access to process ancestry context through the in-memory ancestry acceleration index and the entity extraction caches, enabling chain-scoped rule evaluation at inline latency.

Stage 3: Post-Analysis Response Engine. The third stage executes after events have been written to the graph database and analyzed by the machine-learning-based analyzer. At this stage, allow and block rules are evaluated with full graph database context, including the complete process ancestry chain as stored in the graph. The response engine uses the same rule matching logic and chain pattern language as Stage 2 but derives chain context from authoritative graph queries rather than in-memory caches. This stage handles rules of all types, including those that reference finding titles generated by the analyzer.

3.2 Chain-Filter Pattern Language

The chain-filter pattern language provides a formal syntax for specifying hierarchical process ancestry constraints. A pattern is a sequence of tokens separated by the directional separator `>` (greater-than character), with optional whitespace surrounding each separator. Each token specifies a constraint on one or more elements of the process ancestry chain.

3.2.1 Token Types

The pattern language defines four token types:

1. **Named Token.** A string specifying a process name or user identity. Named tokens are matched against chain elements using case-insensitive glob matching (the `fnmatch` algorithm), supporting the standard glob wildcards `*` (any sequence of characters within a single name), `?` (any single character), and `[seq]` (any character in sequence). Example: `python*` matches `python`, `python3`, and `python3.11`.

2. **Single Wildcard Token (`*`).** When used as a standalone token between separators, the single asterisk matches exactly one process in the chain, regardless of its name. This is distinct from its use within a named token where it serves as an intra-name glob character.

3. **Multi Wildcard Token (`**`).** The double asterisk matches zero or more consecutive processes in the chain. This token enables patterns to match chains of variable depth. Example: `** > rsync > bash` matches any chain ending with `rsync > bash`, regardless of how many ancestor processes precede `rsync`.

4. **User Identity Prefix (`USER:`).** A token prefixed with `USER:` matches against the operating system user identity associated with the process chain rather than a process name. The user identity is prepended to the chain during reconstruction and can be targeted with glob syntax. Example: `USER:www-data > python > curl` matches the chain only when the process is owned by the user `www-data`.

3.2.2 Anchoring Semantics

Patterns are **end-anchored**: the last token of the pattern must match the last element of the process ancestry chain (the triggering process itself). Patterns are **start-unanchored**: the first token of the pattern may match at any position in the chain, not necessarily the root ancestor. This anchoring scheme reflects the operational reality that operators typically care about the terminal portion of a chain (the behavior being allowed or blocked) and its immediate ancestry, but not necessarily the entire chain back to the init process.

Example: The pattern `bash > ncat` matches the chain `[launchd, Terminal, zsh, bash, ncat]` because the pattern matches starting at the `bash` position and consumes through the terminal `ncat` element.

3.2.3 Matching Algorithm

Pattern matching is performed by a recursive backtracking algorithm. The algorithm iterates over all possible starting positions in the chain (implementing the start-unanchored semantics) and, for each starting position, recursively matches pattern tokens against chain elements.

The recursive matching procedure operates as follows:

Base case: If the pattern is exhausted (all tokens consumed), a match is declared if and only if the chain is also exhausted (the end-anchored constraint is satisfied).

Multi-wildcard case (`**`): The algorithm attempts to consume zero, one, two, and up to N remaining chain elements (where N is the number of remaining chain elements), recursively matching the remainder of the pattern against each

resulting suffix. This implements the zero-or-more semantics through exhaustive backtracking.

Single-wildcard case (`*`): The algorithm requires at least one remaining chain element, consumes exactly one element (regardless of its name), and recursively matches the remainder.

Named token case: The algorithm requires at least one remaining chain element, tests whether the current chain element matches the named token using case-insensitive glob matching, and upon match recursively processes the remainder.

Failure: If none of the above cases succeed, the algorithm returns failure for the current starting position and proceeds to the next.

3.3 Two-Tier Evaluation Engine

The two-tier evaluation engine is the central computational mechanism of the invention. It achieves inline enforcement with chain-aware rule evaluation by partitioning rules at compilation time into two distinct evaluation paths based on whether the rule specifies a process ancestry chain filter.

3.3.1 Rule Compilation and Automatic Segregation

Upon initialization and at periodic refresh intervals (configurable, with a default of five seconds), the engine loads all rules from a local persistent store (an SQLite database) and any network-distributed rules received from a fleet management server. Each rule comprises a rule type (one of: `process_name`, `dst_ip`, `dst_cidr`, `domain`, `file_path`, `finding_title`, `chain_pattern`), a pattern string, an optional human-readable description, and an optional chain filter string.

During compilation, each rule is inspected for the presence of a non-empty chain filter. This inspection constitutes the automatic segregation mechanism:

- **Rules without a chain filter** are compiled into type-specific data structures optimized for the fastest possible lookup for that rule type. These structures are collectively referred to as the "fast-path buckets."
- **Rules with a chain filter** are collected into a separate list referred to as the "scoped rules" list. These rules are never placed into the fast-path buckets, regardless of their rule type.

This segregation is a critical correctness property: if a scoped rule were placed into a fast-path bucket, it would be evaluated without chain context and could produce false-positive matches (triggering on the flat attribute alone when the chain context would have excluded the match). The automatic segregation ensures that scoped rules are always evaluated with full chain context.

3.3.2 Tier 1: Fast-Path Bucket Structures

The fast-path buckets comprise the following type-specific data structures:

Data Structure	Rule Type	Lookup Method	Complexity
IP address hash set	`dst_ip`	Exact membership test	O(1) amortized
Domain name hash set	`domain`	Exact membership test (case-normalized)	O(1) amortized

CIDR prefix list	`dst_cidr`	Sequential prefix containment test	O(c) where c = CIDR rule count
Process name glob list	`process_name`	Sequential glob match (fnmatch)	O(p) where p = pattern count
File path glob list	`file_path`	Sequential glob match (fnmatch)	O(f) where f = pattern count
Chain pattern list	`chain_pattern`	Recursive ancestry match	O(k * m) where k = pattern count, m = chain depth

3.3.3 Tier 1 Evaluation Sequence

During evaluation, the engine processes extracted entities from the telemetry event against the fast-path buckets in the following deterministic sequence, returning immediately upon the first match (short-circuit evaluation):

1. **IP Address Check.** For each network connection edge in the extracted entities, test the destination IP address against the IP hash set. If not found, test against each CIDR prefix in the CIDR list.
2. **Domain Name Check.** For each domain in the extracted entities, normalize to lowercase and test against the domain hash set.
3. **Process Name Check.** For each process in the extracted entities, test the process name against each glob pattern in the process name list using case-insensitive fnmatch.
4. **File Path Check.** For each file operation edge in the extracted entities, test the file path against each glob pattern in the file path list.
5. **Chain Pattern Check.** For each process in the extracted entities, reconstruct the process ancestry chain from the in-memory index and test against each chain pattern using the recursive backtracking algorithm.

If no match is found in any fast-path bucket, evaluation proceeds to Tier 2.

3.3.4 Tier 2: Scoped Rule Evaluation with Chain Context

For each rule in the scoped rules list, the engine performs the following steps:

1. **Chain Reconstruction.** For each process in the extracted entities, the engine reconstructs the process ancestry chain by invoking the dual-source chain reconstruction procedure (described in Section 3.5). The resulting chain is a list of string tokens in root-to-leaf order, with the operating system user identity (if known) as the first element, ancestor process names in descending order, and the current process name as the terminal element.
2. **Dual-Condition Evaluation.** The scoped rule is evaluated using a shared matching function that imposes two conditions that must both be satisfied:
    - **Chain Filter Condition.** The reconstructed process ancestry chain must match the rule's chain filter pattern using the recursive backtracking algorithm described in Section 3.2.3.
    - **Rule-Type-Specific Condition.** The entity attribute corresponding to the rule type (process name, IP address, domain name, CIDR range, or file path) must match the rule's pattern using the matching

method appropriate to the rule type (exact match for IPs and domains, CIDR containment for CIDR ranges, glob match for process names and file paths, recursive chain match for chain patterns).

A scoped rule fires if and only if both conditions are simultaneously satisfied. If the chain filter condition fails, the rule is skipped regardless of whether the rule-type-specific condition would have matched.

3. **Finding Synthesis.** Upon a match at either tier, the engine synthesizes a deterministic security finding with critical severity, attaches the matched value, evidence event identifiers, reconstructed chain, affected process identifiers, and relevant indicators of compromise. The finding is dispatched to the response action pipeline for immediate enforcement (for example, process termination), bypassing the asynchronous machine-learning analysis stage.

3.3.5 Thread Safety and Refresh Mechanism

The compiled rule structures are protected by a double-checked locking pattern using a mutual exclusion lock. The engine tracks the last refresh time using a monotonic clock (immune to wall-clock adjustments) and recompiles rules when either the refresh interval has elapsed or an explicit invalidation signal has been received (for example, after an operator adds or removes a rule via the management API, or after new network-distributed rules are received from the fleet server). The invalidation signal sets a boolean flag that is checked on the next evaluation cycle, triggering recompilation under the lock.

3.4 In-Memory Process Ancestry Acceleration Index (PidIndex)

The PidIndex is an in-memory data structure that enables sub-millisecond process ancestry chain reconstruction without querying the graph database. The graph database (an embedded Kuzu instance) indexes nodes by their primary key (a composite string identifier) but does not maintain a secondary index on the integer process identifier (PID) field. Consequently, PID-based lookups against the graph require full table scans, which are prohibitively expensive for inline enforcement where the graph may contain hundreds of thousands of process nodes.

3.4.1 Data Structures

The PidIndex maintains four concurrent hash maps protected by a single mutual exclusion lock:

1. **PID-to-Node-IDs Map** (`pid_to_ids`): Maps each integer PID to a list of graph node identifiers. Multiple node identifiers per PID accommodate PID reuse across different process lifespans (distinguished by the epoch component of the composite node identifier, formatted as `hostname:pid:epoch`).

2. **Parent-to-Children Map** (`ppid_to_children`): Maps each parent PID to the set of child PIDs it has spawned. This structure enables breadth-first traversal of the process tree for descendant resolution.

3. **PID-to-Parent Map** (`pid_to_ppid`): Maps each PID to its parent PID. This structure is the primary data source for upward ancestry chain walking during chain reconstruction.

4. **PID-to-Name Map** (`pid_to_name`): Maps each PID to its process name string. This structure avoids the need to query the graph database or operating system to resolve process names during chain reconstruction.

3.4.2 Bootstrap and Incremental Maintenance

The PidIndex is bootstrapped by a one-time scan of the graph database upon system initialization. The bootstrap query retrieves the node identifier, PID, parent PID, and process name for every Process node in the graph. The results are used to populate all four hash maps atomically under the lock.

After bootstrap, the index is incrementally maintained through a callback mechanism. Each time the graph builder inserts or updates a Process node in the graph database, it invokes the PidIndex callback with the node identifier, PID,

parent PID, and process name. The callback atomically updates all four hash maps under the lock. This incremental maintenance ensures the index reflects the latest process state without requiring periodic full scans.

Stale entries are evicted through a garbage collection mechanism invoked by the graph reaper when it removes expired process nodes from the graph database. The reaper provides a list of node identifiers to remove, and the PidIndex removes those identifiers from the PID-to-Node-IDs map, cleaning up empty PID entries and associated mappings in the other three structures.

3.5 Dual-Source Chain Reconstruction

The chain reconstruction procedure, invoked during Tier 2 evaluation, builds the process ancestry chain for a given process by walking upward through parent process identifiers. The procedure employs a dual-source lookup strategy to maximize coverage and resilience:

Primary source: PidIndex. If the PidIndex has been bootstrapped (indicated by a built flag), the procedure queries the PidIndex for the parent PID and process name of each ancestor.

Fallback source: Entity extraction caches. If the PidIndex has not been bootstrapped, or if a particular lookup fails in the PidIndex, the procedure falls back to in-memory caches populated during the entity extraction phase of event processing. These caches comprise a parent-PID cache (mapping PIDs to parent PIDs), a name cache (mapping PIDs to process names), and a username cache (mapping PIDs to operating system usernames).

The procedure maintains a set of visited PIDs to detect and terminate cycles that may arise from PID reuse or corrupted telemetry. The walk terminates when the parent PID is zero (the init process), the parent PID is not found in either source, or a cycle is detected.

The resulting chain is constructed in the following order:

1. If a username is known for the triggering process, the string `USER:` concatenated with the username is prepended as the first element.
2. Ancestor process names are appended in root-to-leaf order (the earliest ancestor first).
3. The triggering process name is appended as the terminal element.

3.6 Rule Schema and Persistence

Rules are persisted in an SQLite database operating in Write-Ahead Logging (WAL) mode for concurrent read/write safety, with a busy timeout of five seconds. Two tables store allowlist and blocklist rules respectively, sharing an identical schema:

Column	Type	Constraints	Description
`id`	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique rule identifier
`rule_type`	TEXT	NOT NULL	One of seven defined types (see below)
`pattern`	TEXT	NOT NULL, max 500 characters	Matching pattern appropriate to the rule type
`description`	TEXT	DEFAULT empty string	Operator-provided description
`chain_filter`	TEXT	DEFAULT empty string	Optional process ancestry chain pattern

Supported Rule Types:

Rule Type	Pattern Semantics	Matching Method
`process_name`	Process name glob	Case-insensitive fnmatch
`dst_ip`	IP address	Exact string equality
`dst_cidr`	CIDR notation range	IP address prefix containment
`domain`	Domain name	Case-insensitive exact equality
`file_path`	File system path glob	fnmatch
`finding_title`	Security finding title glob	fnmatch (Stage 3 only)
`chain_pattern`	Process ancestry chain	Recursive backtracking (Section 3.2.3)

Rules are deduplicated by the composite key `(rule_type, pattern, chain_filter)`, permitting the same base pattern to exist with different chain filter scopes as distinct rules.

3.7 Threat Intelligence Rule Compilation with Safety Mechanism

The system includes an automated rule compiler that translates external threat intelligence rules (specifically, rules in the Sigma generic signature format published by the SigmaHQ project) into the native chain-aware rule format.

Automatic Chain Filter Generation. When a Sigma rule specifies both an `Image` field (the child process) and a `ParentImage` field (the parent process), the compiler automatically generates a chain filter of the form `** > [ParentImage pattern] > [Image pattern]`, converting the flat Sigma detection into a chain-aware enforcement rule.

Compilation Safety Mechanism (Blast Shield). The compiler incorporates a safety mechanism that prevents the compilation of unscoped rules (rules without a chain filter) that target a predefined set of operating-system-critical process names. These critical names include common shell interpreters (`bash` , `sh`), scripting runtimes (`python` , `perl`), and system utilities (`rm` , `awk` , `sed`). An unscoped block rule targeting any of these names would cause catastrophic false-positive enforcement by blocking legitimate operating system operations. The safety mechanism automatically drops such rules during compilation. Rules targeting these process names are permitted only when accompanied by a non-empty chain filter that constrains the block to a specific ancestry context.

3.8 Network-Distributed Rule Management

Both the allowlist and blocklist enforcement engines support network-distributed rules received from a centralized fleet management server. Network-distributed rules are merged with locally-stored rules during the compilation phase, applying the same two-tier evaluation semantics including automatic segregation of scoped rules. This enables centralized policy management across a fleet of endpoints while preserving the chain-aware matching guarantees of the local enforcement engine.

BRIEF DESCRIPTION OF THE DRAWINGS

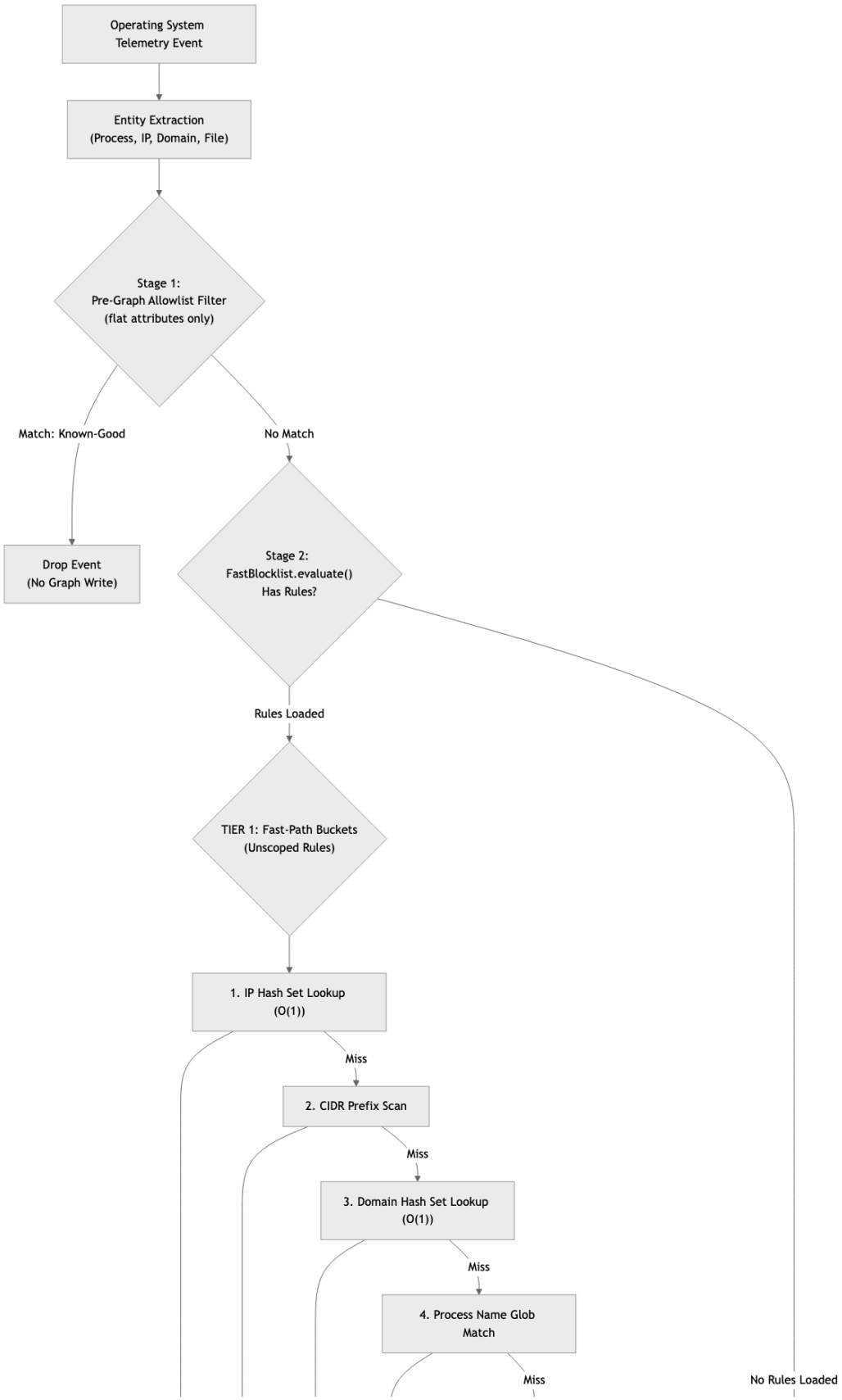
FIG. 1 is a flowchart illustrating the two-tier evaluation pipeline of the present invention, showing the sequential evaluation of operating system telemetry events through the Pre-Graph Allowlist Filter (Stage 1), the Fast-Path Blocklist with Tier 1 constant-time bucket matching and Tier 2 scoped ancestry chain matching (Stage 2), and the Post-Analysis Response Engine (Stage 3).

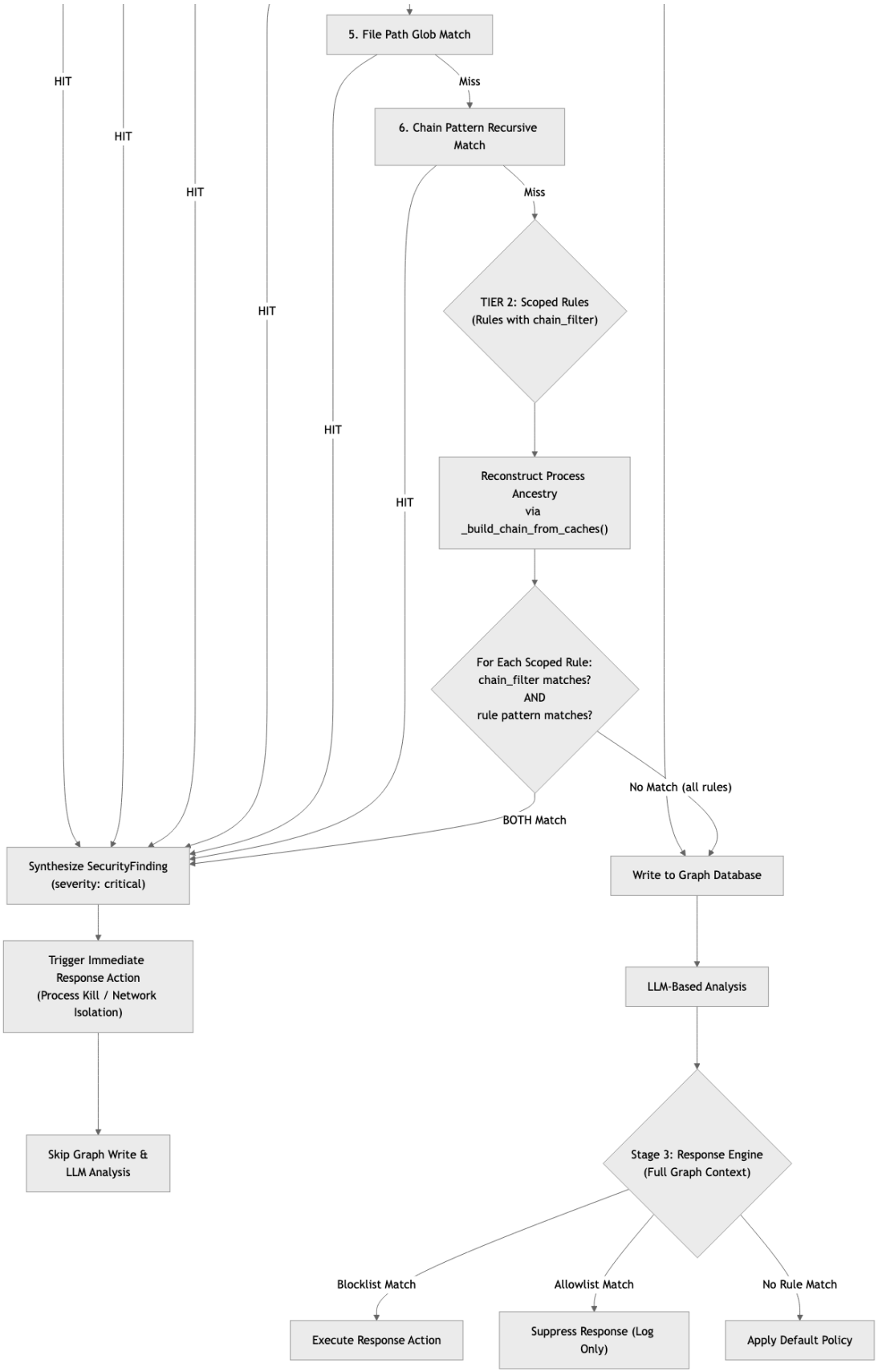
FIG. 2 is a flowchart illustrating the process ancestry chain reconstruction algorithm and the recursive backtracking pattern matching procedure, showing the dual-source parent PID resolution from the in-memory PidIndex and entity extraction caches, the chain assembly with user identity prefix, and the start-unanchored, end-anchored recursive matching logic supporting named tokens, single wildcards, and multi-wildcards.

---

4. SYSTEM FLOW DIAGRAMS

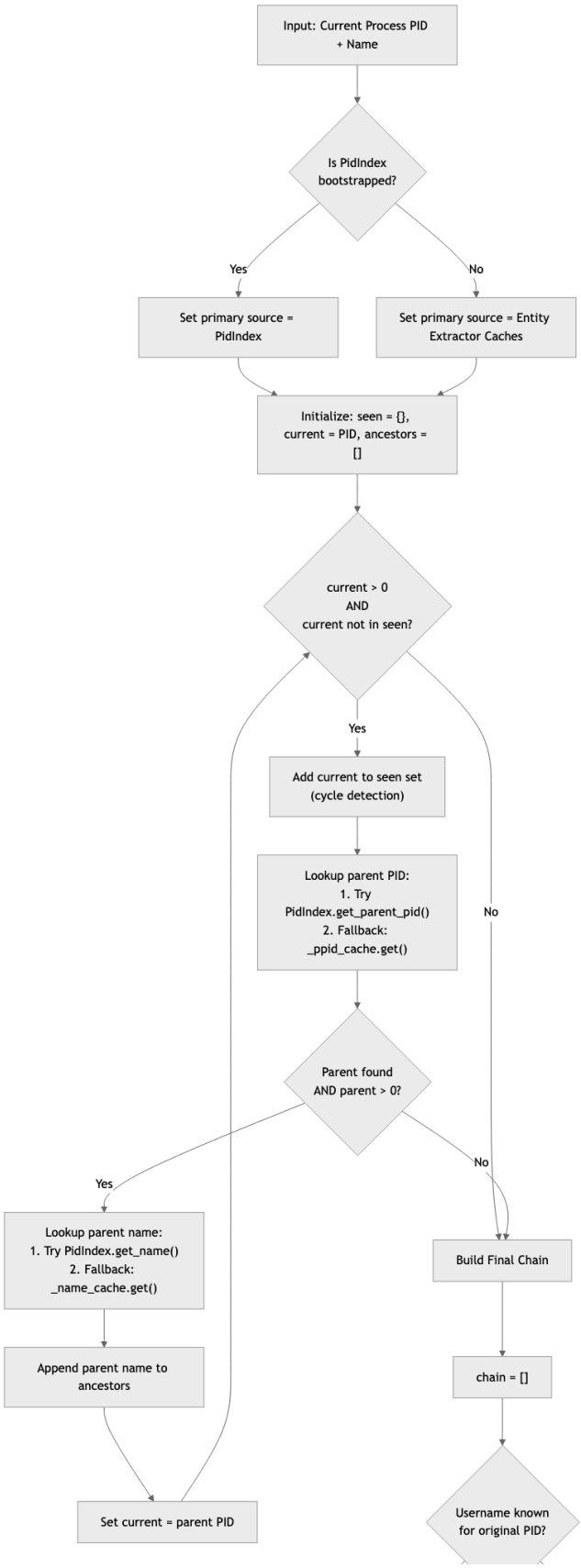
FIG. 1 -- Two-Tier Evaluation Pipeline

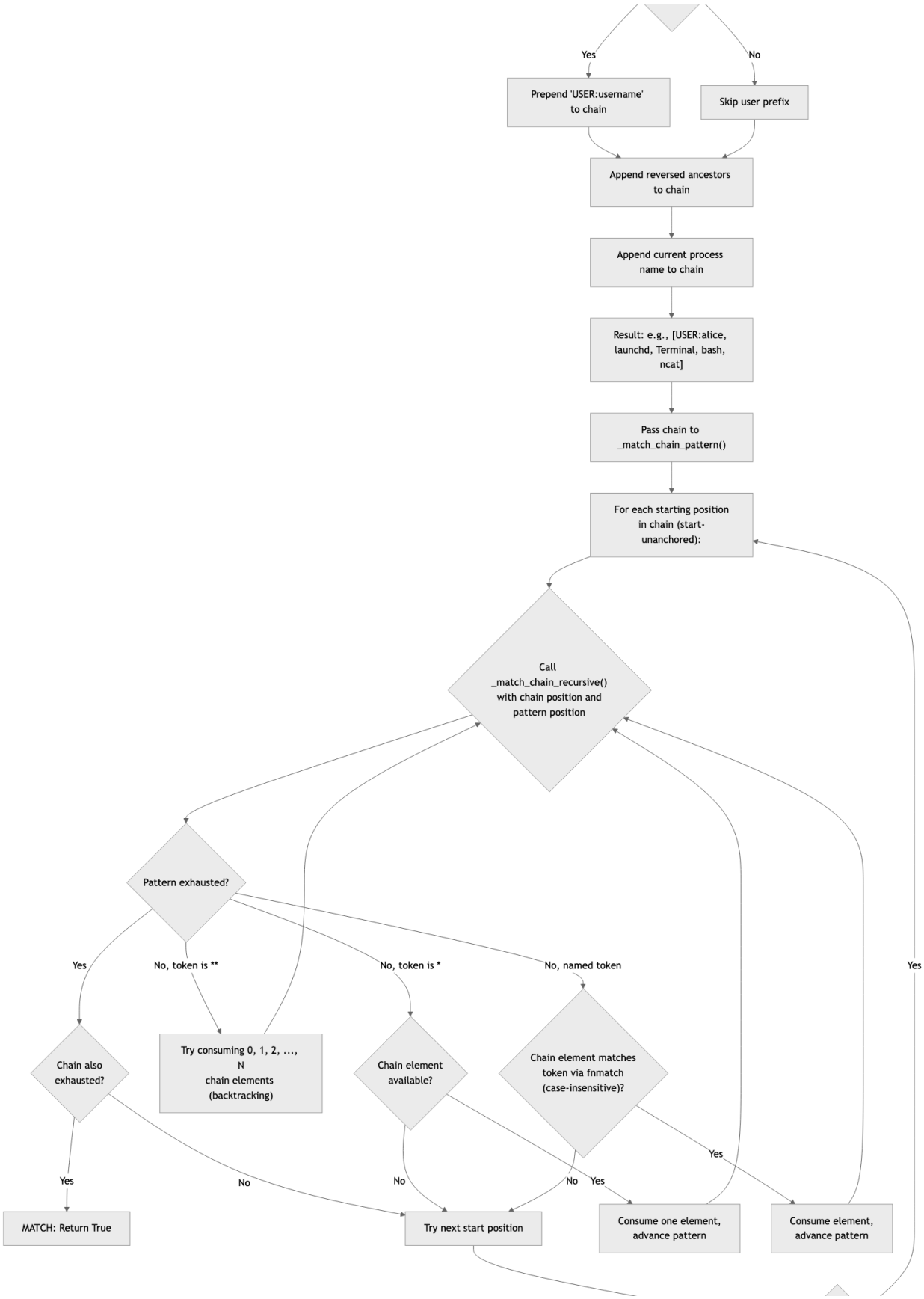

Operating System
Telemetry Event
Entity Extraction
(Process, IP, Domain, File)
Stage 1:
Pre-Graph Allowlist Filter
(flat attributes only)
Match: Known-Good
No Match
Drop Event
(No Graph Write)
Stage 2:
FastBlocklist.evaluate()
Has Rules?
Rules Loaded
TIER 1: Fast-Path Buckets
(Unscoped Rules)
1. IP Hash Set Lookup
(O(1))
Miss
2. CIDR Prefix Scan
Miss
3. Domain Hash Set Lookup
(O(1))
Miss
4. Process Name Glob
Match
Miss
No Rules Loaded

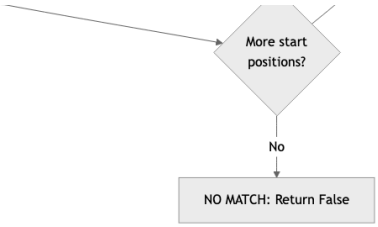

5. File Path Glob Match
HIT
Miss
6. Chain Pattern Recursive Match
HIT
Miss
HIT
TIER 2: Scoped Rules (Rules with chain_filter)
HIT
Reconstruct Process Ancestry via _build_chain_from_caches()
HIT
For Each Scoped Rule: chain_filter matches? AND rule pattern matches?
No Match (all rules)
BOTH Match
Synthesize SecurityFinding (severity: critical)
Write to Graph Database
Trigger Immediate Response Action (Process Kill / Network Isolation)
LLM-Based Analysis
Skip Graph Write & LLM Analysis
Stage 3: Response Engine (Full Graph Context)
Blocklist Match
Allowlist Match
No Rule Match
Execute Response Action
Suppress Response (Log Only)
Apply Default Policy

▶ Mermaid source for FIG. 1

FIG. 2 -- Process Ancestry Chain Reconstruction and Matching Algorithm


Input: Current Process PID + Name
Is PidIndex bootstrapped?
Yes
No
Set primary source = PidIndex
Set primary source = Entity Extractor Caches
Initialize: seen = {}, current = PID, ancestors = []
current > 0 AND current not in seen?
Yes
No
Add current to seen set (cycle detection)
Lookup parent PID: 1. Try PidIndex.get_parent_pid() 2. Fallback: _ppid_cache.get()
Parent found AND parent > 0?
Yes
No
Lookup parent name: 1. Try PidIndex.get_name() 2. Fallback: _name_cache.get()
Append parent name to ancestors
Set current = parent PID
Build Final Chain
chain = []
Username known for original PID?


Yes
No
Prepend 'USER:username' to chain
Skip user prefix
Append reversed ancestors to chain
Append current process name to chain
Result: e.g., [USER:alice, launchd, Terminal, bash, ncat]
Pass chain to _match_chain_pattern()
For each starting position in chain (start-unanchored):
Call _match_chain_recursive() with chain position and pattern position
Pattern exhausted?
Yes
No, token is **
No, token is *
No, named token
Chain also exhausted?
Try consuming 0, 1, 2, ..., N chain elements (backtracking)
Chain element available?
Chain element matches token via fnmatch (case-insensitive)?
Yes
No
No
No
Yes
Yes
MATCH: Return True
Try next start position
Consume one element, advance pattern
Consume element, advance pattern
Yes



▶ Mermaid source for FIG. 2

---

5. DRAFT CLAIMS

Claim 1 (Independent -- Method)

A computer-implemented method for enforcing endpoint security rules in real-time on a computing device, the method comprising:

(a) receiving a stream of operating system telemetry events at an endpoint agent executing on the computing device, wherein said telemetry events are sourced from one or more kernel-level instrumentation mechanisms including Extended Berkeley Packet Filter (eBPF) tracepoints and kprobes, operating system audit subsystems, Endpoint Security Framework (ESF) event subscriptions, or Event Tracing for Windows (ETW) kernel provider sessions;

(b) extracting, from each telemetry event, entity attributes comprising at least one of: a process identity including a process identifier and process name, a network endpoint including a destination IP address, a domain name, and a file system path;

(c) compiling a plurality of user-configurable enforcement rules from a persistent rule store into a first set and a second set, wherein each rule comprises a rule type, a pattern, and an optional process ancestry chain filter, and wherein the compiling step segregates rules specifying a non-empty chain filter into the second set and rules not specifying a chain filter into the first set;

(d) evaluating the extracted entity attributes against the first set of rules using type-specific data structures optimized for constant-time or near-constant-time lookup, wherein said evaluation does not require reconstruction of the process ancestry chain;

(e) upon no match in step (d), evaluating the extracted entity attributes against the second set of rules, wherein said evaluation comprises, for each rule in the second set:

(i) reconstructing a process ancestry chain for the triggering process by iteratively resolving parent process identifiers from an in-memory process ancestry index, the chain comprising an ordered sequence of ancestor process names from root ancestor to triggering process;

(ii) matching the reconstructed process ancestry chain against the rule's chain filter pattern using a recursive backtracking algorithm that supports named tokens matched by case-insensitive glob matching, a single-wildcard token matching exactly one chain element, a multi-wildcard token matching zero or more consecutive chain elements, and a user identity prefix token;

(iii) matching the extracted entity attribute corresponding to the rule type against the rule's pattern using a type-appropriate matching method; and

(iv) determining that the rule is satisfied if and only if both the chain filter pattern match of step (ii) and the rule-type-specific pattern match of step (iii) succeed;

(f) upon a match in step (d) or step (e), generating a deterministic security finding and triggering an automated response action on the computing device without awaiting completion of asynchronous machine-learning-based analysis of the telemetry event.

Claim 2 (Dependent on Claim 1)

The method of Claim 1, wherein the type-specific data structures of step (d) comprise:

a hash set data structure for destination IP address lookup providing O(1) amortized-time membership testing;

a hash set data structure for domain name lookup providing O(1) amortized-time membership testing with case normalization;

a prefix list data structure for CIDR range containment testing; and

ordered glob pattern lists for process name matching and file path matching using the fnmatch algorithm.

Claim 3 (Dependent on Claim 1)

The method of Claim 1, wherein the process ancestry chain filter pattern of step (ii) employs a pattern language comprising:

a directional token separator character between process name elements;

a single-asterisk token that, when used as a standalone element between separators, matches exactly one process in the ancestry chain;

a double-asterisk token that matches zero or more consecutive processes in the ancestry chain; and

a user identity prefix that scopes the match to a specific operating system user identity associated with the process chain.

Claim 4 (Dependent on Claim 1)

The method of Claim 1, wherein the chain filter pattern of step (ii) is end-anchored at the terminal process of the ancestry chain such that the last token of the pattern must match the triggering process, and is start-unanchored such that the first token of the pattern may match at any ancestor position in the chain.

Claim 5 (Dependent on Claim 1)

The method of Claim 1, wherein the in-memory process ancestry index of step (i) is:

bootstrapped by a one-time scan of a graph database storing process relationship data, the scan populating hash map structures mapping process identifiers to parent process identifiers and process names; and

incrementally maintained by a callback invoked upon each process node insertion into the graph database, the callback atomically updating the hash map structures under a mutual exclusion lock.

Claim 6 (Dependent on Claim 1)

The method of Claim 1, wherein the reconstruction step (i) employs a dual-source lookup strategy comprising:

a primary lookup against the in-memory process ancestry index; and

a fallback lookup against in-process entity extraction caches populated during the entity extraction step (b), the fallback being invoked when the primary lookup fails or when the in-memory index has not yet been bootstrapped;

wherein a cycle detection mechanism maintains a set of visited process identifiers and terminates the ancestry walk upon detecting a previously visited identifier.

Claim 7 (Independent -- System)

A system for chain-aware endpoint detection and response enforcement, comprising:

a processor module configured to receive operating system telemetry events and extract entity attributes therefrom;

a persistent rule store comprising a database with columns for rule type, pattern, description, and chain filter, wherein rules are deduplicated by a composite key of rule type, pattern, and chain filter;

an in-memory process ancestry index maintaining hash map structures mapping process identifiers to parent process identifiers, child process identifier sets, graph database node identifiers, and process names, the index being bootstrapped from a graph database and incrementally maintained through process lifecycle callbacks;

a fast-path enforcement engine that, upon initialization and at periodic refresh intervals, compiles rules from the persistent rule store into segregated data structures, wherein rules specifying a non-empty chain filter are automatically segregated into a scoped-rules list separate from type-specific constant-time lookup structures; and

a two-tier evaluation pipeline that, for each incoming telemetry event, first evaluates entity attributes against unscoped rules using the constant-time lookup structures, and upon no match evaluates against scoped rules by reconstructing process ancestry from the in-memory index and requiring simultaneous satisfaction of both the chain filter pattern and the rule-type-specific pattern.

Claim 8 (Dependent on Claim 7)

The system of Claim 7, further comprising a rule compiler configured to translate threat intelligence rules from an external generic signature format into the native rule format, wherein the compiler automatically generates a chain filter pattern when both a child-process field and a parent-process field are present in a source rule, constructing the chain filter in the form of a multi-wildcard token followed by a directional separator, the parent-process pattern, another directional separator, and the child-process pattern.

Claim 9 (Dependent on Claim 7)

The system of Claim 7, wherein the rule compiler further comprises a compilation safety mechanism that maintains a predefined set of operating-system-critical process names and prevents compilation of any rule that targets a process name in said predefined set unless the rule specifies a non-empty chain filter, thereby preventing catastrophic false-positive enforcement against essential operating system processes.

Claim 10 (Dependent on Claim 7)

The system of Claim 7, further comprising a network distribution mechanism wherein a centralized fleet management server transmits rules to a plurality of endpoint agents, and each endpoint agent merges the received network-distributed rules with locally-stored rules during the compilation phase, applying the same two-tier evaluation semantics including automatic segregation of scoped rules to both local and network-distributed rules.

Claim 11 (Independent -- Computer-Readable Medium)

A non-transitory computer-readable storage medium storing instructions that, when executed by one or more processors of a computing device, cause the computing device to perform the method of Claim 1.

DISCLAIMER

This document constitutes a draft provisional patent application prepared for technical review purposes. It does not constitute legal advice. The claims, specifications, and descriptions herein should be reviewed and refined by a registered patent attorney or patent agent before filing with the United States Patent and Trademark Office. Provisional patent applications establish a priority date but do not mature into issued patents without the filing of a corresponding non-provisional application within twelve months.

Prepared: February 2026 Inventors: [TO BE SPECIFIED] Assignee: [TO BE SPECIFIED]